SOFTWARE ENGINEERING 1

MODULE 7

DESIGN AND IMPLEMENTATION

Technology	Purpose
Vue.js + Vite	Frontend application framework and project tooling
Tailwind CSS	Responsive interface styling
JavaScript	Application and CRUD logic
localStorage	Browser-based prototype data persistence
Git + GitHub	Version control and repository submission
GitHub Actions	Simple continuous-integration build check

Prepared by: Patrick Jason L. Torres

School of Computer Studies

1. Activity Overview

In Module 6, you prepared an architectural design for a proposed information system. In this activity, you will translate one selected module or entity from that architecture into a working frontend prototype.

> **Scope rule:** Implement only one manageable entity. You are not required to build the entire proposed system.

The prototype will use Vue.js, Tailwind CSS, JavaScript, and browser localStorage. The complete backend, API, and database shown in your Module 6 architecture may remain **proposed future components**.

2. Learning Outcomes

- Connect an architectural design to an actual software implementation.
- Create and organize a Vue.js website using reusable components.
- Apply Tailwind CSS classes to build a responsive user interface.
- Implement Create, Read, Update, Delete, search, and validation functions.
- Save and retrieve prototype data using browser localStorage.
- Manage project versions using Git and GitHub.
- Verify that the Vue application builds successfully through GitHub Actions.

3. Required Output

Output	Minimum Requirement
Simple system	One entity selected from the Module 6 design
Frontend	Vue.js components styled with Tailwind CSS
Functions	Add, view, edit, delete, search, and validation
Persistence	Records remain after page refresh through localStorage
Repository	Public GitHub repository with meaningful commits
CI	Successful GitHub Actions production-build check
Evidence	PDF report, screenshots, repository link, and ZIP file

4. Selecting a Module from Module 6

Review your Module 6 README and architecture document. Choose a record type that has clear fields and can support CRUD operations.

Proposed System	Recommended Entity	Example Fields
Library System	Books	Book ID, title, author, category, status
Inventory System	Products	Product ID, name, category, quantity, price

Proposed System	Recommended Entity	Example Fields
Asset System	Equipment	Asset code, name, location, condition
Appointment System	Appointments	Client, date, time, purpose, status
Student System	Students	Student number, name, program, year level

Approval rule: The selected entity must belong to the system proposed in Module 6. Do not change to an unrelated project topic.

5. Relationship Between Modules 6 and 7

Module 6 Element	Module 7 Implementation
Proposed complete system	Basis and long-term blueprint
Presentation layer	Vue components and Tailwind CSS interface
System module/entity	One selected functional prototype
User interactions	Forms, buttons, record list, and search
Application logic	JavaScript CRUD and validation functions
Data layer	Simulated using browser localStorage
Backend/API/database	Future implementation; not required now

6. Functional and Interface Requirements

6.1 Required Functions

Create - Add a complete and valid record using a form.

Read - Display all records in a table, card list, or organized layout.

Update - Load an existing record, change its information, and save it.

Delete - Remove a record only after user confirmation.

Search - Filter records using at least one relevant field.

Validation - Prevent submission when required fields are empty.

Persistence - Keep records available after refreshing the browser.

6.2 Required Interface Sections

- Application header or navigation bar
- System title and short description
- Record entry and edit form

- Search field
- Record table or card list
- Edit and Delete action buttons
- Success, warning, or error feedback
- Record count or simple summary
- Footer with student name and section

6.3 Vue Component Requirement

Create at least three reusable Vue components. Use meaningful names based on the selected system.

```
src/
|-- components/
|   |-- AppHeader.vue
|   |-- RecordForm.vue
|   |-- RecordList.vue
|   `-- AppFooter.vue
|-- App.vue
|-- main.js
`-- style.css
```

7. Guided Technical Procedure

Step 1. Prepare the design-to-code plan

Write the selected entity, fields, operations, and component names. Sketch the form and record-list layout before coding.

Planning Item	Student Entry
Proposed system	_____
Selected entity	_____
Required fields	_____
Vue components	_____
Search field	_____

Step 2. Verify required software

```
node --version
npm --version
git --version
```

Each command should display an installed version number. If a command is not recognized, review the installation lesson before continuing.

Step 3. Create the Vue project

```
npm create vite@latest surname-module7-system -- --template vue
cd surname-module7-system
npm install
npm run dev
```

Expected result: Vite displays a local address such as http://localhost:5173/. Open it in a browser and confirm that the Vue starter page appears.

Step 4. Install Tailwind CSS v4

```
npm install tailwindcss @tailwindcss/vite
```

In vite.config.js, import the Tailwind Vite plugin and add it to the plugins array:

```
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [vue(), tailwindcss()],
})
```

Replace the contents of src/style.css with:

```
@import "tailwindcss";
```

> **Important:** This guide uses Tailwind CSS v4. Older tutorials may show a different tailwind.config.js procedure.

Step 5. Clean and organize the starter project

Remove unused starter content, create the components folder, and prepare the component files required by your chosen system.

Step 6. Build the interface

Use Tailwind utility classes to create the header, form, search field, responsive record list, status messages, and footer.

Step 7. Create the reactive record state

```
import { ref, computed, onMounted } from 'vue'

const records = ref([])
const searchTerm = ref('')
const editingId = ref(null)
```

Step 8. Load records from localStorage

```
onMounted(() => {
  const saved = localStorage.getItem('module7-records')
  records.value = saved ? JSON.parse(saved) : []
})

function saveRecords() {
  localStorage.setItem(
    'module7-records',
    JSON.stringify(records.value)
  )
}
```

Step 9. Implement Create

```
function addRecord(newRecord) {
  records.value.push({
    id: Date.now(),
    ...newRecord
  })
  saveRecords()
}
```

Step 10. Implement Delete with confirmation

```
function deleteRecord(id) {
  const confirmed = window.confirm(
    'Are you sure you want to delete this record?'
  )

  if (!confirmed) return

  records.value = records.value.filter(
    record => record.id !== id
  )
  saveRecords()
}
```

Step 11. Implement Search

```
const filteredRecords = computed(() => {
  const keyword = searchTerm.value.toLowerCase().trim()

  return records.value.filter(record =>
    record.name.toLowerCase().includes(keyword)
  )
})
```

> **Customization:** Replace record.name with the field appropriate to your selected entity, such as title, productName, studentName, or equipmentName.

Step 12. Implement Edit and Update

Load the selected record into the form, store its ID, update the matching array item, call saveRecords(), and return the form to Add mode.

Step 13. Add validation and feedback

Show a clear message when fields are incomplete and show success feedback after adding or updating a record.

Step 14. Verify responsive design

Check the application on desktop and smaller browser widths. Forms, buttons, and records must remain readable and usable.

8. Testing Checklist

Test	Expected Result
Add a complete record	Record appears and success feedback is shown
Submit an empty required field	Submission is prevented
Add multiple records	All records display correctly
Edit a record	Updated values replace the previous values
Cancel deletion	Record remains in the list
Confirm deletion	Selected record is removed
Search an existing value	Matching records are displayed
Search a missing value	Empty-state message appears

Test	Expected Result
Refresh the browser	Saved records remain available
Resize the browser	Interface remains usable

9. Git and GitHub Workflow

Step 1. Initialize Git and make the first commit

```
git init
git add .
git commit -m "Create initial Vue project"
```

Step 2. Create a public GitHub repository

Use the exact repository format: surname-module7-vue-system.

Step 3. Connect and push the project

```
git branch -M main
git remote add origin YOUR_REPOSITORY_URL
git push -u origin main
```

Step 4. Repeat the development workflow

```
git status
git add .
git commit -m "Describe the completed change"
git push
```

9.1 Required Commit History

Create at least five meaningful commits. Recommended examples:

- Create initial Vue project
- Configure Tailwind CSS
- Create reusable system components
- Implement CRUD and localStorage
- Add search and form validation
- Improve responsive interface
- Complete README and documentation

Avoid: Do not use unclear messages such as update, changes, final, done, or 123.

10. Continuous Integration Build Check

Create the following file inside the project:

```
.github/
`-- workflows/
    `-- build.yml
```

Paste this workflow into build.yml:

```
name: Vue Build Check

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm

      - name: Install dependencies
        run: npm ci

      - name: Build Vue application
        run: npm run build
```

Commit and push the workflow:

```
git add .
git commit -m "Add GitHub Actions build workflow"
git push
```

Expected result: Open the Actions tab of the repository. The Vue Build Check workflow must complete successfully and display a green check mark.

11. README Documentation

The README.md file must contain:

- Project title, student name, and section
- System description and selected Module 6 entity
- List of implemented features
- Technologies used
- Installation and run instructions
- Explanation of localStorage
- Connection between Module 6 and Module 7
- Application screenshots
- Known limitations and proposed future improvements

12. Required Screenshot Evidence

Filename	Required Evidence
01-running-application.png	Complete application running in the browser
02-add-record.png	Completed form and newly added record
03-record-list.png	Multiple records displayed
04-edit-record.png	Existing record being edited
05-delete-confirmation.png	Delete confirmation message
06-search-function.png	Working search or filter
07-localstorage.png	Saved data shown in browser developer tools
08-responsive-view.png	Application at a smaller screen width
09-github-repository.png	Public repository and project files
10-commit-history.png	At least five meaningful commits
11-ci-success.png	Successful GitHub Actions workflow